

Documentação de Requisitos Implementados

Projeto MotherClean

Integrante: Guilherme Carvalho Alves

Data: Novembro de 2025

1. Comunicação com Banco de Dados

Implementação

O sistema utiliza SQLite como banco de dados e implementa 4 entidades principais com relacionamentos:

Entidades:

1. Usuario

- Campos: username, email, password, data_cadastro, ultimo_acesso
- Modelo customizado baseado em AbstractUser do Django
- Localização: core/models.py

2. Componente

- Campos: nome, descricao, instrucoes_limpeza, imagem_url, video_youtube, ordem_exibicao, ativo
- Representa cada peça do computador
- Localização: core/models.py

3. ProgressoUsuario

- Campos: usuario, componente, visualizado, data_visualizacao
- Relacionamentos: ForeignKey para Usuario e Componente
- Registra quais componentes cada usuário visualizou
- Localização: core/models.py

4. TarefaManutencao

- Campos: usuario, componente, descricao, data_agendada, concluida, data_conclusao
- Relacionamentos: ForeignKey para Usuario e Componente
- Gerencia cronograma de manutenções

- Localização: core/models.py

Relacionamentos:

Usuario (1) -----> (N) ProgressoUsuario

Usuario (1) -----> (N) TarefaManutencao

Componente (1) --> (N) ProgressoUsuario

Componente (1) --> (N) TarefaManutencao

Operações CRUD Implementadas:

CREATE (Criar):

- Cadastro de usuário: cadastro_view em views.py
- Criar progresso: componente_detalhe_view em views.py
- Criar tarefas: gerar_cronograma_view em views.py
- Criar componentes: Via Django Admin

READ (Ler):

- Listar componentes: home_view em views.py
- Ler detalhes do componente: componente_detalhe_view em views.py
- Listar tarefas: cronograma_view em views.py
- Consultar progresso: home_view em views.py

UPDATE (Atualizar):

- Atualizar último acesso: login_view em views.py
- Marcar tarefa como concluída: tarefa_concluir_view em views.py
- Atualizar componentes: Via Django Admin

DELETE (Deletar):

- Deletar tarefas: tarefa_deletar_view em views.py
- Deletar componentes: Via Django Admin

Arquivo do banco de dados: db.sqlite3 na raiz do projeto

2. Aparência e Responsividade

Implementação

O sistema utiliza tecnologias modernas para criar uma interface responsiva e interativa:

Tecnologias Utilizadas:

CSS:

- Arquivo: static/css/style.css
- Sistema de variáveis CSS para cores consistentes
- Layouts responsivos com CSS Grid e Flexbox
- Media queries para adaptação mobile
- Animações e transições suaves
- Gradientes e efeitos visuais modernos

JavaScript:

- Arquivo: static/js/main.js
- Auto-hide de mensagens após 5 segundos
- Confirmação antes de deletar tarefas
- Estados de loading em botões
- Smooth scroll para navegação
- Fetch API para consumir endpoint de clima

Imagens:

- Imagens dos componentes armazenadas em static/images/
- Placeholder images para componentes sem foto
- Ícones emoji para interface amigável

Responsividade:

O sistema é totalmente responsivo e se adapta a diferentes tamanhos de tela:

Desktop (> 768px):

- Layout em duas colunas para gabinete e lista
- Sidebar de clima integrado no header
- Hotspots posicionados sobre a imagem do gabinete

Mobile (< 768px):

- Layout em coluna única
- Menu de navegação verticalizado
- Cards de tarefas empilhados

Teste de responsividade:

1. Abra o site no navegador
 2. Pressione F12 (DevTools)
 3. Clique no ícone de dispositivo móvel
 4. Teste em diferentes resoluções
-

3. Implantação em Produção

Status: PENDENTE

4. Controle de Versão**Implementação**

Plataforma: GitHub

Repositório: <https://github.com/GuilhermeCarvalho/aplicacao-web-mother-clean---ufsc>

Estrutura do repositório:

- Código-fonte completo do projeto
- README com instruções de instalação
- Histórico de commits documentando o desenvolvimento
- Branches organizadas (se aplicável)

Como verificar:

1. Acessar o link do repositório
 2. Navegar pelos arquivos do projeto
 3. Verificar commits e histórico de alterações
-

5. Rastreamento de Usuários**Implementação**

Ferramenta: Microsoft Clarity

Localização: Integrado no template base (templates/base.html)

Como funciona:

O Microsoft Clarity rastreia:

- Número de visitantes e sessões
- Mapas de calor (heatmaps) mostrando onde usuários clicam
- Gravações de sessões (como usuários navegam)
- Métricas de performance e comportamento

Código implementado:

```
<!-- Microsoft Clarity -->
```

```
<script type="text/javascript">
```

```
(function(c,l,a,r,i,t,y){  
    c[a]=c[a]||function(){(c[a].q=c[a].q||[]).push(arguments)};  
    t=l.createElement(r);t.async=1;t.src="https://www.clarity.ms/tag/"+i;  
    y=l.getElementsByTagName(r)[0];y.parentNode.insertBefore(t,y);  
})(window, document, "clarity", "script", "ID_DO_PROJETO");
```

```
</script>
```

Localização no código: templates/base.html (dentro da tag <head>)

Como verificar:

1. Acessar <https://clarity.microsoft.com/>
2. Fazer login com a conta Microsoft
3. Visualizar projeto "MotherClean"
4. Verificar dados de rastreamento:
 - Dashboard com métricas
 - Gravações de sessões
 - Mapas de calor

Observação: O rastreamento só funciona quando o site está sendo acessado (mesmo localmente). Dados aparecem no painel do Clarity após alguns minutos de uso.

6. Páginas de Erro

Implementação

Páginas personalizadas para erros HTTP 404 e 500.

Arquivos criados:

1. Página 404 (Página não encontrada)

- Template: templates/404.html
- Design personalizado com ícone de busca
- Código de erro grande e visível
- Mensagem explicativa amigável
- Botões para voltar ao início ou página anterior

2. Página 500 (Erro interno do servidor)

- Template: templates/500.html
- Design personalizado com ícone de alerta
- Mensagem informando sobre o erro
- Botões para voltar ao início ou tentar novamente

Configuração:

Views de erro: core/views.py

```
def handler404(request, exception):  
    return render(request, '404.html', status=404)
```

```
def handler500(request):  
    return render(request, '500.html', status=500)
```

Handlers configurados: mc/urls.py

```
handler404 = 'core.views.handler404'
```

```
handler500 = 'core.views.handler500'
```

Como testar:

Testar 404:

1. Acesse uma URL inexistente: `http://127.0.0.1:8000/pagina-que-nao-existe/`
2. A página 404 personalizada deve aparecer

Testar 500:

1. Temporariamente force um erro em alguma view
2. Ou acesse: `http://127.0.0.1:8000/teste-500/` (se criou URL de teste)
3. A página 500 personalizada deve aparecer

Observação: Em modo `DEBUG=True`, o Django mostra a página de debug padrão. Para ver as páginas personalizadas, configure `DEBUG=False` no `settings.py`.

7. Testes Automatizados

Implementação Completa

Arquivo: `core/tests.py`

Total de testes: 26 testes automatizados

Framework: Django TestCase (baseado em unittest)

Categorias de Testes:

7.1 Testes de Models (4 testes)

Classe: `UsuarioModelTest`

- `test_criar_usuario`: Verifica criação de usuário com username, email e senha
- `test_usuario_string_representation`: Testa representação em string do modelo

Classe: `ComponenteModelTest`

- `test_criar_componente`: Verifica criação de componente com todos os campos
- `test_componente_string_representation`: Testa representação em string

O que testam:

- Criação correta de objetos no banco de dados
- Integridade dos dados salvos
- Métodos de representação dos modelos

7.2 Testes de Relacionamentos (2 testes)

Classe: ProgressoUsuarioModelTest

- test_criar_progresso: Verifica criação de registro de progresso
- test_progresso_unique_constraint: Testa restrição de unicidade (não permitir duplicatas)

Classe: TarefaManutencaoModelTest

- test_criar_tarefa: Verifica criação de tarefa com data agendada
- test_marcar_tarefa_concluida: Testa método de conclusão de tarefa

O que testam:

- Relacionamentos entre tabelas (Foreign Keys)
- Constraints de banco de dados
- Métodos customizados dos modelos

7.3 Testes de Autenticação (7 testes)

Classe: AutenticacaoViewsTest

- test_pagina_login_get: Verifica acesso à página de login
- test_login_com_credenciais_validas: Testa login bem-sucedido
- test_login_com_credenciais_invalidas: Testa login com senha errada
- test_pagina_cadastro_get: Verifica acesso à página de cadastro
- test_cadastro_usuario_valido: Testa cadastro com dados corretos
- test_cadastro_senhas_diferentes: Testa validação de senhas
- test_logout: Verifica processo de logout

O que testam:

- Fluxo completo de autenticação
- Validações de formulários
- Redirecionamentos corretos
- Mensagens de erro apropriadas
- Criação de sessões de usuário

7.4 Testes de Views Protegidas (3 testes)

Classe: HomeViewTest

- test_home_requer_login: Verifica que home redireciona usuários não autenticados
- test_home_com_usuario_logado: Testa acesso autorizado à home

Classe: ComponenteDetalheViewTest

- test_componente_detalhe_requer_login: Verifica proteção da página de detalhes
- test_componente_detalhe_registra_progresso: Testa registro automático de progresso

O que testam:

- Decorator @login_required funcionando
- Acesso autorizado vs não autorizado
- Registro automático de dados
- Renderização correta de templates

7.5 Testes de Funcionalidades (2 testes)

Classe: CronogramaViewTest

- test_cronograma_requer_login: Verifica proteção da página
- test gerar_cronograma: Testa geração automática de tarefas

O que testam:

- Criação automática de tarefas baseadas em progresso
- Lógica de negócio do cronograma
- Integração entre modelos

7.6 Testes CRUD Completos (4 testes)

Classe: CRUDOperacoesTest

- test_crud_create: Testa operação CREATE
- test_crud_read: Testa operação READ
- test_crud_update: Testa operação UPDATE
- test_crud_delete: Testa operação DELETE

O que testam:

- Todas as operações CRUD funcionando

- Persistência de dados
- Atualização de registros
- Remoção de registros

Como executar os testes:

Rodar todos os testes:

```
python manage.py test
```

Saída esperada:

```
Creating test database for alias 'default'...
```

```
System check identified no issues (0 silenced).
```

```
.....
```

```
-----
```

```
Ran 25 tests in 7.964s
```

```
OK
```

```
Destroying test database for alias 'default'...
```

Rodar testes específicos:

```
# Testar apenas um app
```

```
python manage.py test core
```

```
# Testar apenas uma classe
```

```
python manage.py test core.tests.AuthenticacaoViewsTest
```

```
# Testar apenas um método
```

```
python manage.py test
```

```
core.tests.AuthenticacaoViewsTest.test_login_com_credenciais_validas
```

```
# Rodar com mais detalhes (verboso)
```

```
python manage.py test --verbosity=2
```

Estrutura de um teste (exemplo):

```
def test_login_com_credenciais_validas(self):  
    # 1. Arrange (Preparar)  
    # Usuário já criado no setUp()  
  
    # 2. Act (Agir)  
    response = self.client.post(reverse('login'), {  
        'username': 'testuser',  
        'password': 'senha123'  
    })  
  
    # 3. Assert (Verificar)  
    self.assertEqual(response.status_code, 302) # Redirect  
    self.assertRedirects(response, reverse('home'))
```

Cada teste segue o padrão AAA (Arrange, Act, Assert):

1. **Arrange:** Preparar dados e estado inicial
2. **Act:** Executar a ação que está sendo testada
3. **Assert:** Verificar se o resultado está correto

8. Autenticação e Autorização

Implementação

Sistema próprio de autenticação implementado com Django Auth.

Componentes:

1. Modelo de Usuário Customizado

- Arquivo: core/models.py
- Classe: Usuario(AbstractUser)
- Campos adicionais: data_cadastro, ultimo_acesso

- Configurado em: settings.py → AUTH_USER_MODEL = 'core.Usuario'

2. Views de Autenticação

Cadastro: cadastro_view

- Validação de senhas coincidentes
- Verificação de username/email duplicados
- Criação segura de usuário com hash de senha
- Localização: core/views.py

Login: login_view

- Autenticação com username e senha
- Registro de último acesso
- Criação de sessão
- Localização: core/views.py

Logout: logout_view

- Encerramento de sessão
- Redirecionamento para login
- Localização: core/views.py

3. Proteção de Rotas

Todas as páginas principais utilizam @login_required:

- home_view
- componente_detalhe_view
- cronograma_view
- gerar_cronograma_view
- tarefa_concluir_view
- tarefa_deletar_view
- clima_view

4. Configurações de Segurança

Em settings.py:

LOGIN_URL = 'login'

LOGIN_REDIRECT_URL = 'home'

LOGOUT_REDIRECT_URL = 'login'

5. Templates de Autenticação

- templates/login.html
- templates/cadastro.html

Fluxo de Autenticação:

1. Usuário acessa o site
2. Se não autenticado, é redirecionado para /login/
3. Após login bem-sucedido, é redirecionado para / (home)
4. Todas as páginas internas requerem autenticação
5. Logout redireciona para login

Segurança Implementada:

- Senhas hashadas com PBKDF2 (padrão Django)
- Proteção CSRF em todos os formulários
- Validação de dados no backend
- Sessões seguras
- Mensagens de erro genéricas (não revelam se username existe)

9. Integração com API Externa

Implementação

API Utilizada: OpenWeatherMap API (Clima)

Endpoint: <http://api.openweathermap.org/data/2.5/weather>

Relevância para o Projeto:

A API de clima foi integrada porque condições ambientais (temperatura e umidade) afetam diretamente a manutenção de computadores:

- Umidade baixa (<40%) → Mais acúmulo de poeira → Aumentar frequência de limpeza
- Umidade alta (>70%) → Risco de condensação → Cuidado com componentes

- Umidade ideal (40-70%) → Condições normais de manutenção

Implementação:

1. View da API: clima_view em core/views.py

@login_required

def clima_view(request):

lat = '-27.5969' # Florianópolis

lon = '-48.5495'

api_key = 'SUA_CHAVE'

url =

f'http://api.openweathermap.org/data/2.5/weather?lat={lat}&lon={lon}&appid={api_key}&units=metric&lang=pt_br'

response = requests.get(url, timeout=5)

data = response.json()

Processa dados e retorna recomendações

...

2. Endpoint criado: /api/clima/

- Configurado em core/urls.py
- Retorna JSON com dados de clima

3. Consumo no Frontend:

- Arquivo: templates/home.html
- Usa Fetch API do JavaScript
- Atualiza DOM dinamicamente

4. Interface:

- Widget visual integrado no header da home
- Exibe: cidade, temperatura, umidade, condição
- Mostra recomendação personalizada baseada na umidade

Dados Retornados:

```
{  
  "temperatura": 21.8,  
  "umidade": 67,  
  "descricao": "nublado",  
  "recomendacao": "✅ Umidade ideal para seus componentes!",  
  "cidade": "Florianópolis"  
}
```

Como testar:

1. Acessar a home logado
2. Widget de clima aparece no topo direito
3. Dados são carregados automaticamente
4. Recomendação muda baseada na umidade atual

Teste direto da API:

<http://127.0.0.1:8000/api/clima/>

Requisitos:

- Biblioteca requests instalada
- Chave de API do OpenWeatherMap configurada
- Conexão com internet ativa

10. Metodologia Ágil (SCRUM)**Implementação**

Ferramenta: Trello

Metodologia: SCRUM adaptado para projeto individual

Estrutura do Trello: <https://trello.com/b/X2zKzlbF/motherclean-dw>